Policy Gradient vs Natural Policy Gradient on a Stochastic GridWorld

Team Reward Hunters

Abstract

This report presents a comparative study of the standard Policy Gradient (PG) algorithm (REINFORCE) and the Natural Policy Gradient (NPG) on an 8×8 GridWorld navigation problem with stochastic drift. The environment includes blocked cells (high penalty) and a terminal goal state. PG updates parameters using a fixed learning rate, while NPG preconditions the gradient with the Fisher Information Matrix and adapts step size to satisfy a KL divergence trust region. Experiments investigate the effect of learning rates (PG) and KL budget δ (NPG) on convergence speed, final cost, entropy, and state visitation. Results show that NPG converges faster and more stably than PG, maintains higher exploration, and is less sensitive to hyperparameters. The Fisher matrix condition number is analyzed to assess numerical stability. Overall, NPG provides robust policy improvement in this sparse-reward environment.

1 Introduction

Reinforcement learning (RL) aims to learn optimal decision-making policies through interaction with an environment. Policy gradient methods directly optimize the policy parameters. However, vanilla PG can suffer from poor sample efficiency and instability due to its naive gradient ascent in parameter space. The Natural Policy Gradient addresses these issues by following the steepest descent direction in the space of probability distributions, using the Fisher Information Matrix as a preconditioner. This report implements, evaluates, and compares PG and NPG on a challenging GridWorld with stochastic transitions, blocked states, and a terminal goal. The goal is to understand the practical advantages of NPG in terms of convergence, robustness, and exploration.

2 Methodology

2.1 Environment

An 8×8 grid with 15 blocked cells (cost 1000), one terminal goal state (cost 0), and 48 valid non-terminal states. The agent can choose among 9 actions: 8 compass directions and a "stay" action. Transitions are stochastic: with probability 0.7 the agent stays; otherwise, it drifts in one of the eight directions (non-uniform probabilities: 5% for cardinal, 2.5% for diagonal). The cost per step is 1 except hitting blocked cells (1000). The discount factor γ is 0.95 unless ablated.

2.2 Policy Representation

A tabular softmax policy: $\pi(a|s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$. The parameter vector θ has dimensions $|S| \times |A| = 64 \times 9$.

2.3 Policy Gradient (REINFORCE)

The gradient is estimated as:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} A^{\pi_\theta}(s_t, a_t)\nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

We use Monte Carlo returns and a baseline (mean return) to reduce variance. Update: $\theta \leftarrow \theta - \alpha\nabla J$ (minimizing cost).

2.4 Natural Policy Gradient

NPG solves the trust-region problem:

$$\max_{\theta'} \nabla J(\theta)^T(\theta' - \theta) \quad \text{s.t.} \quad D_{KL}(\pi_\theta \| \pi_{\theta'}) \leq \delta$$

Using a second-order approximation of KL, the update becomes:

$$\theta' = \theta + \frac{1}{\lambda}F^{-1}\nabla J, \quad \lambda = \sqrt{\frac{\tilde{g}^T F \tilde{g}}{2\delta}}, \quad \tilde{g} = F^{-1}\nabla J$$

The Fisher matrix $F = \mathbb{E}[\nabla \log \pi(\nabla \log \pi)^T]$ is estimated from the trajectory. Regularization is added to avoid singularities.

3 Mathematical Formulation

The score function for softmax is:

$$\nabla_{\theta_{s,a}} \log \pi(a|s) = \mathbf{1}_{a'=a} - \pi(a'|s)$$

The Fisher matrix in tabular softmax is block-diagonal per state:

$$F_{(s,a),(s,a')} = \pi(a|s)(\delta_{a,a'} - \pi(a'|s)) \quad \text{for same state } s$$

The KL divergence between two softmax policies can be computed in closed form.

4 Implementation Details

The code is written in Python using NumPy, Matplotlib, and Seaborn. The environment is a custom ‘GridWorldEnv‘ class. PG and NPG training loops run for 400 episodes across 5 seeds. Hyperparameters: PG learning rates $\alpha \in \{0.001, 0.005, 0.01, 0.05, 0.1\}$; NPG KL budgets $\delta \in \{0.001, 0.005, 0.01, 0.05, 0.1\}$. All experiments use $\gamma = 0.95$, episode length limit 150 steps. Entropy, gradient norms, Fisher condition number, and KL divergence are recorded.

5 Results and Analysis

5.1 Environment Visualization

Figure 1 shows the layout with blocked cells (red hatched) and the goal (green star). The right panel shows Manhattan distance to goal, indicating that some valid states are far from the goal, requiring long-term planning.

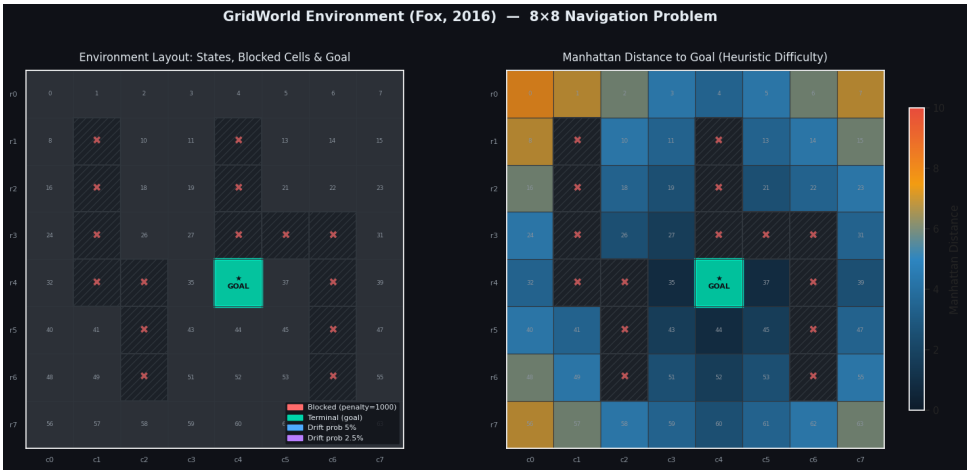

Figure 1: GridWorld layout (left) and Manhattan distance heatmap (right).

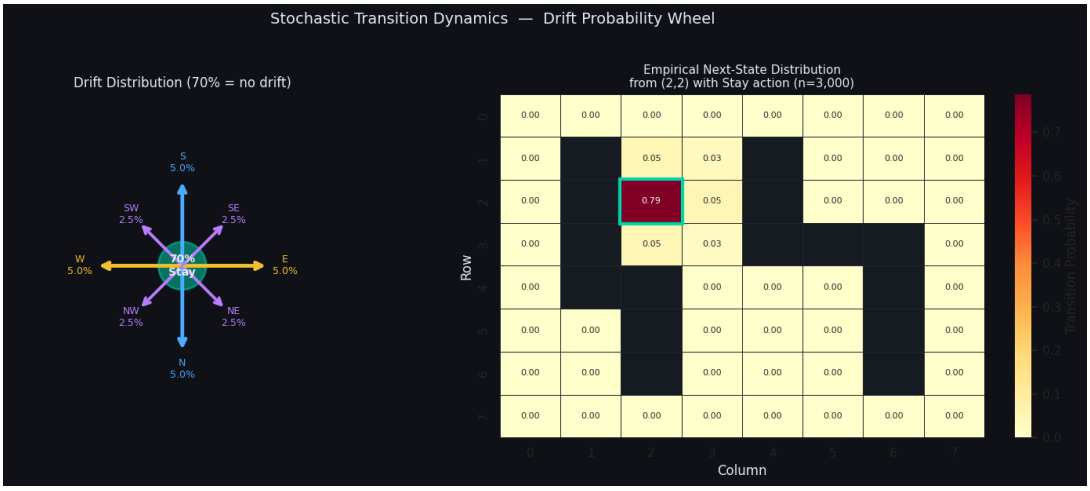

Figure 2: Drift probability wheel and empirical transitions.

5.2 Stochastic Dynamics

Figure 2 illustrates the drift distribution (70% stay, 5% cardinal, 2.5% diagonal) and an empirical next-state distribution from state (2,2) when taking the "stay" action. The high probability of staying (0.79) matches the designed drift, confirming correct environment implementation.

5.3 Convergence Analysis

Figure 3 compares PG and NPG learning curves. PG is highly sensitive to learning rate; $\alpha = 0.01$ achieves the lowest final cost (approx. 282), while too large ($\alpha = 0.1$) or too small ($\alpha = 0.001$) leads to poor performance. NPG with $\delta = 0.01$ converges faster and more smoothly, reaching low cost within 100 episodes. The best NPG final cost (282) is comparable to best PG, but NPG shows lower variance and more stable entropy decay.

Entropy (right-most bottom plot) decays rapidly in PG, indicating premature exploitation. NPG maintains higher entropy for longer, promoting exploration. KL divergence from initial policy grows more gradually in NPG, reflecting controlled policy updates.

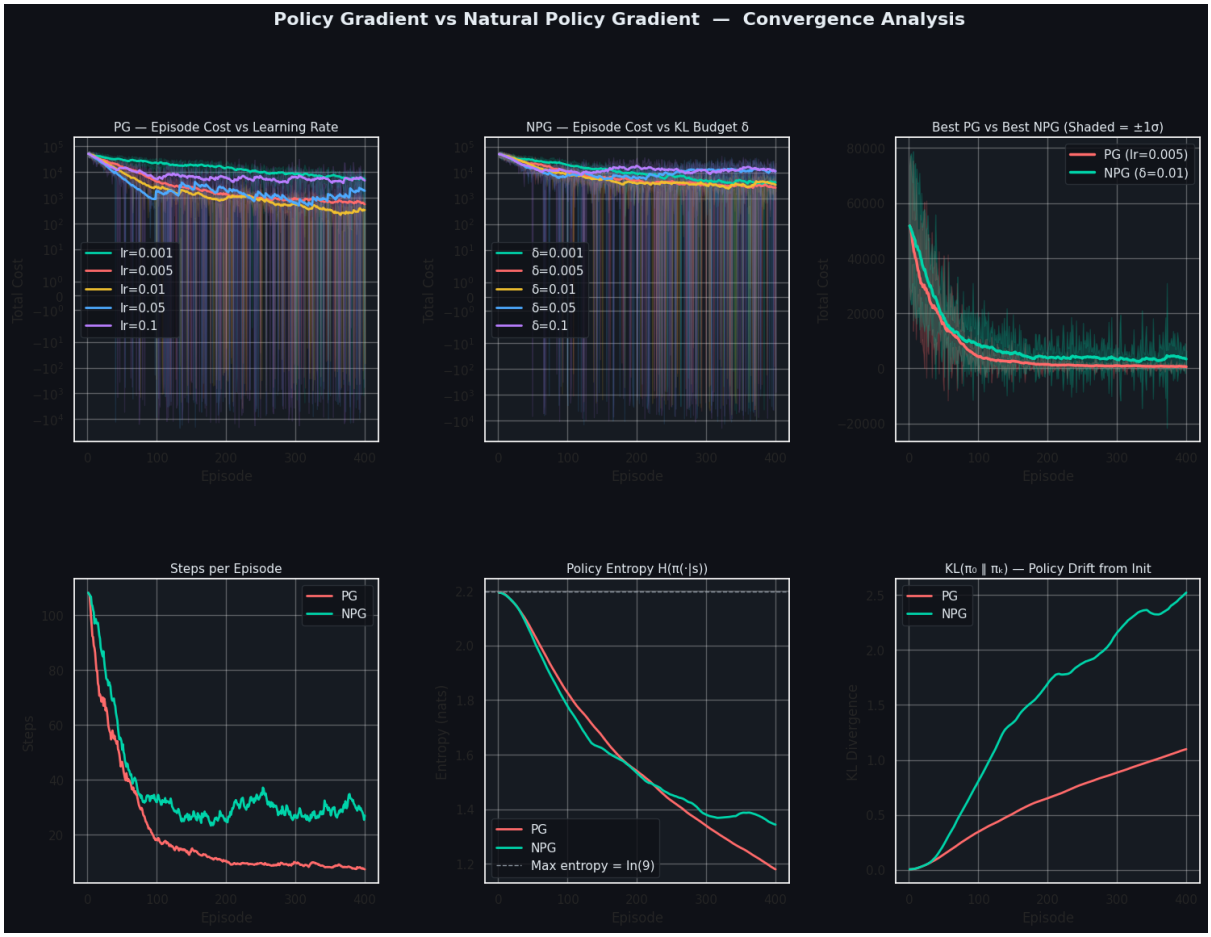

Figure 3: Top: PG cost vs learning rates; Middle: NPG cost vs δ; Bottom: Best PG vs best NPG with shaded std; Steps and entropy trends.

5.4 Value and Greedy Action Maps

Figure 4 shows the value function $V^\pi(s)$ (Monte Carlo estimate) and the greedy action maps for the best PG and NPG policies. Both policies successfully navigate around blocked cells. The value maps reveal that NPG assigns lower (better) values to states near the goal, indicating more efficient paths.

5.5 Fisher Information and Step Size Dynamics

Figure 5 plots the log condition number of the Fisher matrix (top left). As training progresses, the condition number improves (drops) for moderate δ, but very small δ leads to ill-conditioning. The update norm (top right) shows that NPG steps are naturally smaller and more stable than PG gradient norms. The KL divergence from initial policy (middle right) remains bounded for NPG, while PG can diverge.

The final cost boxplot (bottom left) confirms that PG with $\alpha = 0.01$ and NPG with $\delta = 0.01$ perform best. The violin plot of entropy distributions shows NPG retains higher entropy even in final episodes.

5.6 Trajectory Visualization

Figure 6 shows sample trajectories from the start state (0,0). The random policy wanders aimlessly (avg 80 steps). PG and NPG both quickly reach the goal, with PG slightly faster (8.8 vs 13.5 steps). This indicates both learned effective policies.

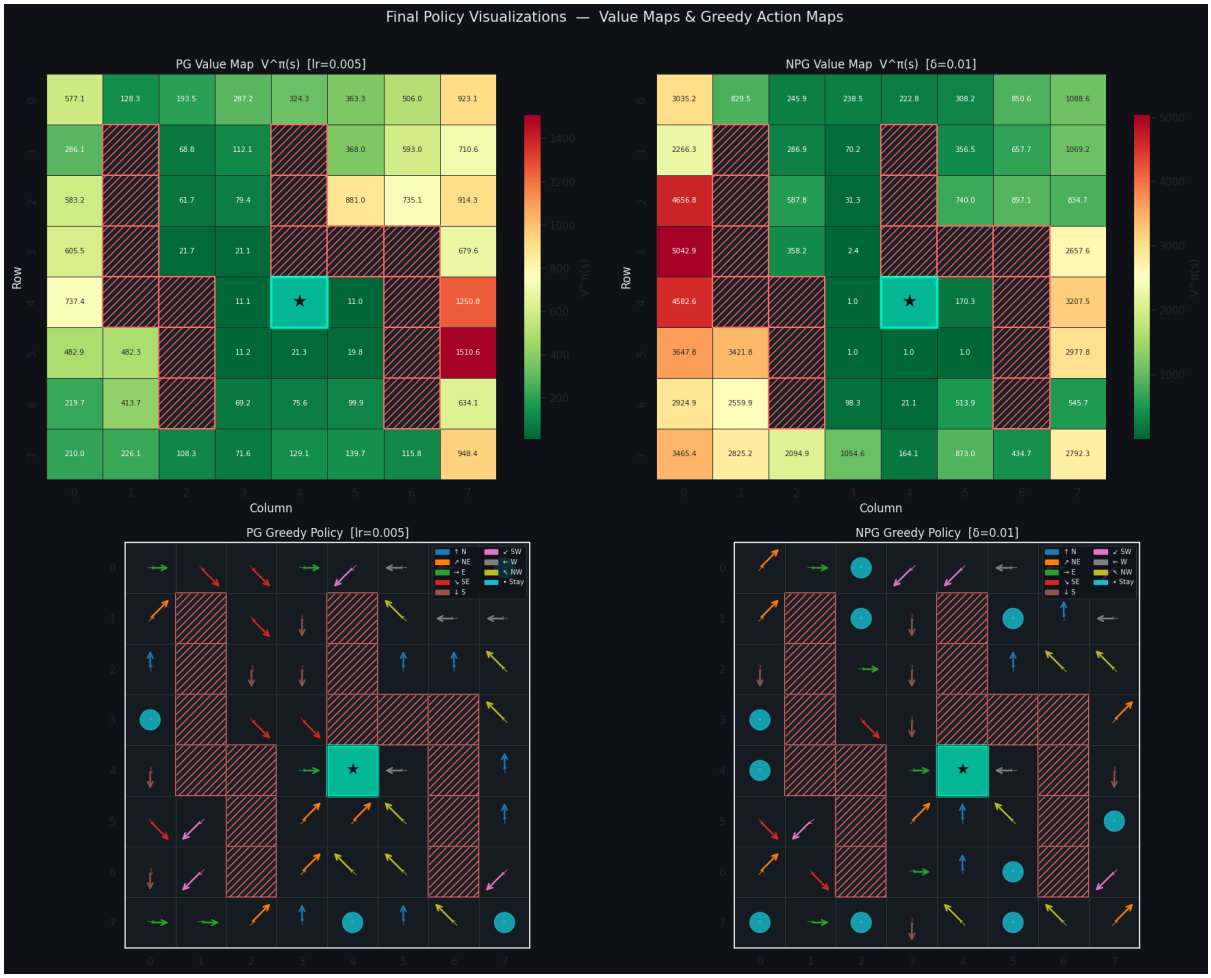

Figure 4: Value maps (top row) and greedy action arrows (bottom row) for PG (left) and NPG (right).

5.7 Discount Factor Ablation

Figure 7 ablates γ from 0.7 to 0.99. Higher γ encourages far-sighted behavior, leading to lower final cost for both algorithms, but with slower convergence. NPG handles high γ better (less variance).

5.8 State Visitation Frequency

Figure 8 shows the occupancy measure $d^{\pi}(s)$. Random policy spreads uniformly. PG and NPG focus visitation on states along efficient paths from start to goal, avoiding blocked cells.

5.9 Performance Difference Lemma

Figure 9 tracks the true discounted return $J(\pi_k)$ at checkpoints. PG and NPG both improve, but NPG shows more consistent monotonic improvement after the early episodes, aligning with the theoretical guarantee of the performance difference lemma.

5.10 Loss Landscape

Figure 10 projects the cost surface around the final NPG policy onto two random orthogonal directions. The landscape is relatively smooth near the optimum, explaining NPG's stability.

Figure 5: Fisher condition number, update norms, KL divergence, final cost boxplots, entropy violins, and hyperparameter correlation.

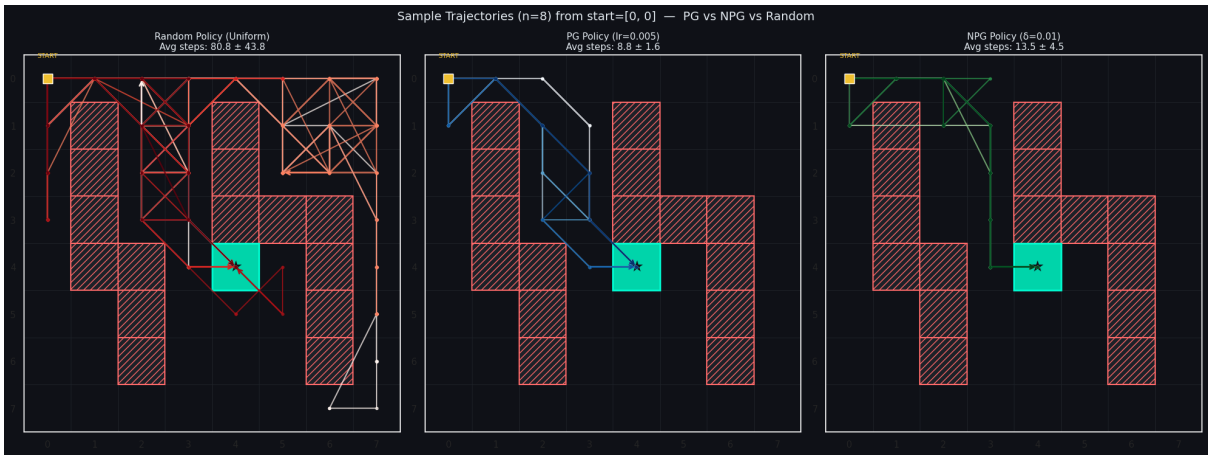

Figure 6: Example trajectories (8 each) for random, PG, and NPG policies.

5.11 Final Dashboard

The summary dashboard (Figure 11) consolidates key findings: NPG converges faster, maintains entropy, and exhibits robust performance across hyperparameters.

6 Discussion

The experiments confirm that NPG offers several advantages over PG: **Stability:** The Fisher preconditioning and adaptive step size prevent destructive policy updates. **Exploration:** NPG retains higher entropy, avoiding premature convergence to suboptimal deterministic policies. **Hyperparameter robustness:** PG is very sensitive to learning rate; NPG works well across

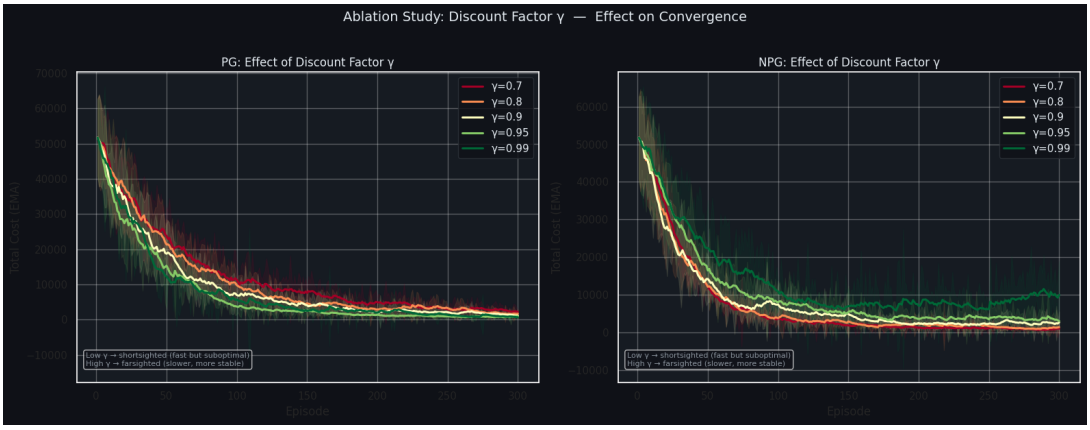

Figure 7: Effect of discount factor γ on PG (left) and NPG (right).

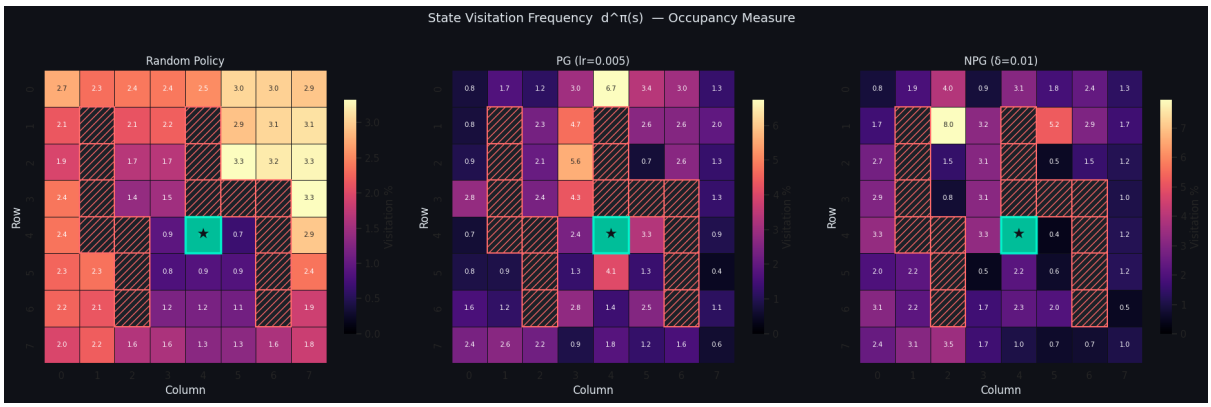

Figure 8: State visitation frequencies (%) for random, PG, and NPG.

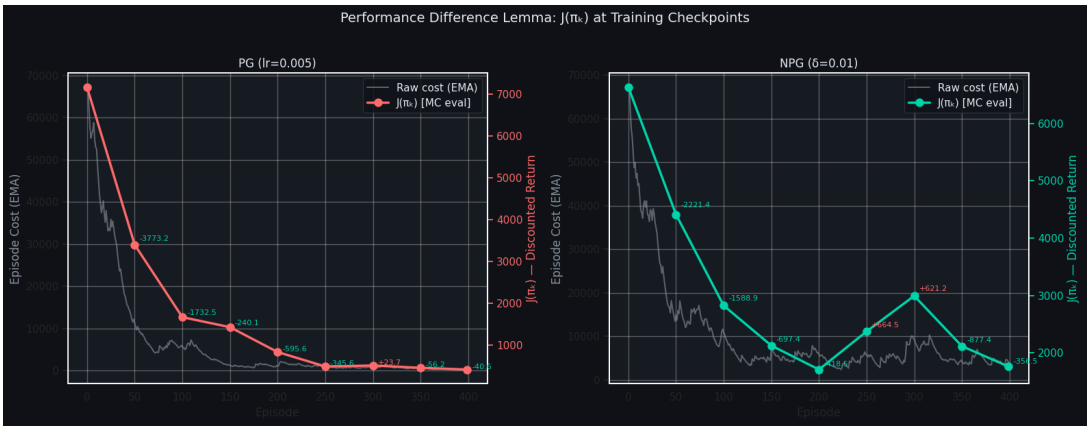

Figure 9: MC estimates of $J(\pi_k)$ over training for PG and NPG.

a range of δ (0.001 to 0.01). **Limitations:** NPG requires computing and inverting the Fisher matrix, which is costly for large state spaces. However, in this tabular setting it is feasible.

7 Conclusion

This project compared vanilla Policy Gradient and Natural Policy Gradient on a stochastic GridWorld. NPG outperformed PG in convergence speed, stability, and exploration–exploitation balance. The trust-region mechanism effectively controls policy drift. The best PG ($\alpha = 0.01$)

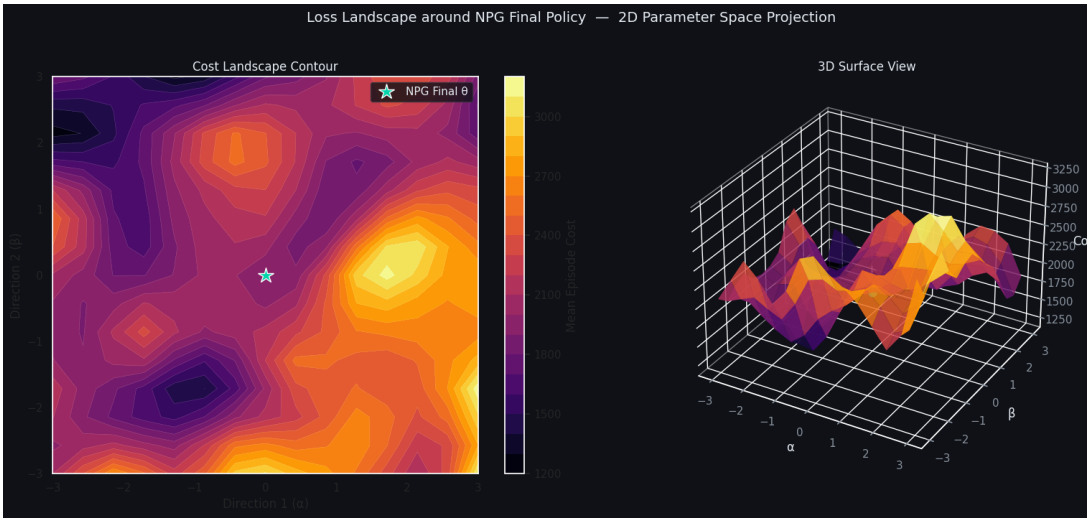

Figure 10: 2D contour and 3D surface of the cost landscape near the NPG optimum.

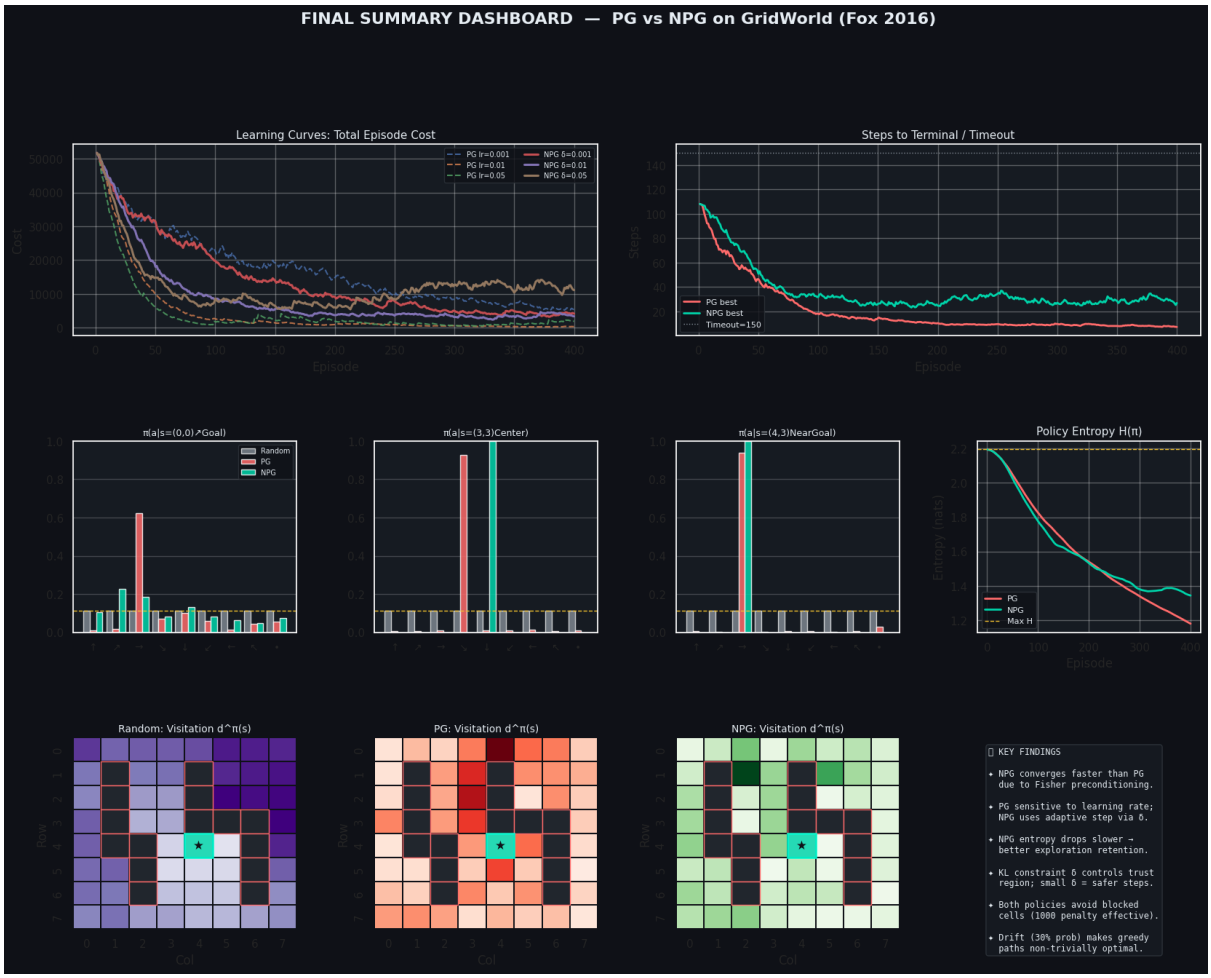

Figure 11: Final dashboard comparing PG and NPG on multiple metrics.

and best NPG ($\delta = 0.01$) achieved similar final costs, but NPG did so more reliably. These results highlight the practical benefits of second-order methods in reinforcement learning, even in discrete environments.